
aliases: - 17-tuple-set

元组 (Tuple) + 集合 (Set)

前置：已掌握 list 和 dict 目标：搞懂元组和集合是什么、怎么用、和 list/dict 的区别 用时：约 10 分钟

相关笔记：[列表深入操作](#) · [字典 dict](#)

一、元组 (Tuple)：不可变的列表

基本操作

```
# 元组用 (), 列表用 []
t = (1, 2, 3)
print(t[0])      # 1 — 和列表一样索引
print(t[1:3])    # (2, 3) — 和列表一样切片

# 遍历
for x in t:
    print(x)

# 长度
print(len(t))    # 3
```

元组 vs 列表

```
# 列表可变
lst = [1, 2, 3]
lst[0] = 99      # ✅ [99, 2, 3]

# 元组不可变
```

```
t = (1, 2, 3)

t[0] = 99      # ❌ TypeError: 'tuple' object does not support item assignment
t.append(4)     # ❌ 没有 append 方法
t.remove(1)    # ❌ 没有 remove 方法
```

元组能做的操作：索引、切片、遍历、`len()`、`count()`、`index()` **元组不能做的操作：**增、删、改

`count()` — 统计某个值出现了几次

```
scores.count(80)    # 3 — 80 分出现了 3 次
scores.count(90)     # 2
scores.count(100)    # 0 — 没有人考 100
```

`index()` — 查找某个值第一次出现在哪个位置

```
scores.index(90)     # 1 — 第一个 90 在索引 1
scores.index(80)     # 0 — 第一个 80 在索引 0
scores.index(100)    # ❌ ValueError: 不在元组里 → 报错
```

为什么需要元组？

```
# 1. 作为字典的键（列表不行）
d = {}
d[("张三", "男")] = 92    # ✅ 元组可哈希
d[["张三", "男"]] = 92    # ❌ TypeError: unhashable type: 'list'

# 2. 函数返回多个值（本质就是元组）
def min_max(nums):
    return min(nums), max(nums) # 返回元组

result = min_max([3, 1, 4, 1, 5])
print(result)           # (1, 5)
print(type(result))     # <class 'tuple'>

# 拆包
low, high = min_max([3, 1, 4, 1, 5])
print(low, high)        # 1 5
```

```
# 3. 表示“不该被修改”的数据
# 坐标、RGB颜色、配置项等
COUNTRY_CAPITAL = (“中国”, “北京”)    # 常量元组
```

[!NOTE]- 我的理解：可哈希 vs 不可哈希

可哈希 = 一辈子固定不变，值永远不改 Python 能给它们算出一个唯一固定编号（哈希值），字典靠这个编号找数据。

可哈希（不可变）	不可哈希（可变）
整数、字符串、布尔、 元组	列表、字典、集合
一旦创建，不能改里面任何元素	随时可以增删改，值会变，哈希值就乱了，字典没法用它当键

元组的特殊语法

```
# 单元素元组——必须加逗号！
t1 = (1)      # 这是 int，不是 tuple
t2 = (1,)     # 这才是元组
print(type(t1)) # <class 'int'>
print(type(t2)) # <class 'tuple'>

# 省略括号（逗号才是关键）
t = 1, 2, 3
print(t)      # (1, 2, 3)
print(type(t)) # <class 'tuple'>

# 拆包
a, b, c = (1, 2, 3)
print(a, b, c) # 1 2 3

# 交换变量（元组拆包的经典用法）
x, y = 10, 20
x, y = y, x    # 等价于 (x, y) = (y, x)
print(x, y)    # 20 10
```

一句话记

元组 = 只读列表。不能增删改，但能做字典键、函数多返回值、拆包。

二、集合 (Set)：无序、不重复

基本操作

```
# 集合用 {}, 和字典一样, 但没有键值对
s = {1, 2, 3, 2, 1}
print(s)          # {1, 2, 3} — 自动去重

# 创建空集合
s = set()          # ✅ 空集合
d = {}             # ❌ 这是空字典, 不是空集合

# 添加和删除
s = {1, 2, 3}
s.add(4)            # {1, 2, 3, 4}
s.add(2)            # {1, 2, 3, 4} — 已存在, 啥也不做
s.remove(3)         # {1, 2, 4}
# s.remove(99)      # ❌ KeyError — 不存在就报错
s.discard(99)       # ✅ 不存在也不报错
```

集合运算

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)        # 并集: {1, 2, 3, 4, 5, 6} 等价 a.union(b)
print(a & b)         # 交集: {3, 4} 等价 a.intersection(b)
print(a - b)         # 差集: {1, 2} 等价 a.difference(b)
print(a ^ b)         # 对称差: {1, 2, 5, 6} 等价 a.symmetric_difference(b)

# 判断
print(1 in a)        # True
print(10 in a)       # False
```

```
print(a.isdisjoint({7, 8})) # True — 没有交集
print({1, 2}.issubset(a))   # True — 子集
```

集合的常见用途

```
# 1. 去重
nums = [1, 2, 2, 3, 3, 3, 4]
unique = list(set(nums))
print(unique)      # [1, 2, 3, 4] (顺序不保证)

# 2. 快速成员检查 (集合的 in 比列表快很多)
names_set = {"张三", "李四", "王五"}
print("张三" in names_set) # True — O(1)

# 3. 找出差异
old_users = {"张三", "李四", "王五"}
new_users = {"张三", "赵六", "钱七"}

added = new_users - old_users      # {'赵六', '钱七'}
removed = old_users - new_users    # {'李四', '王五'}
```

集合推导式

```
nums = [1, 2, 2, 3, 3, 3]
s = {x**2 for x in nums}
print(s) # {1, 4, 9} — 自动去重
```

一句话记

集合 = 无序的不重复元素集。| 并集、& 交集、- 差集。去重和成员检查最快。

备考相关

- [[EXAM_PREP/day02/00_今日任务]] — Day 2 字符串与组合数据类型